ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ
ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ & ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΗΛΕΚΤΡΟΝΙΚΗΣ & ΥΠΟΛΟΓΙΣΤΩΝ — ΕΝΣΩΜΑΤΩΜΕΝΑ ΣΥΣΤΗΜΑΤΑ ΠΡΑΓΜΑΤΙΚΟΥ ΧΡΟΝΟΥ

Πολυνηματικό Ενσωματωμένο Σύστημα Πραγματικού Χρόνου για Ασύγχρονη Λήψη, Επεξεργασία και Σύγχρονη Τηλεμετρία του Bluesky Jetstream Firehose

Πλατφόρμα Εκτέλεσης: Physical Hardware Raspberry Pi Zero W (BCM2835 ARMv6zk @ 1 GHz)

Ονοματεπώνυμο: Ευάγγελος Στεργιόπουλος **ΑΕΜ:** 10711 **GitHub Repository:** https://github.com/VangelisSter/rtes-realtime-telemetry-rpi

1. Εισαγωγή & Προδιαγραφές Προβλήματος

Σκοπός της παρούσας εργασίας είναι ο σχεδιασμός, η βέλτιστη υλοποίηση σε γλώσσα C (POSIX user-space) και η πειραματική αξιολόγηση ενός πολυνηματικού ενσωματωμένου συστήματος πραγματικού χρόνου (RTES) χαμηλού κόστους και περιορισμένων πόρων (Raspberry Pi Zero W). Το σύστημα υλοποιεί:

1. **Ασύγχρονη λήψη (event-driven)** δεδομένων τηλεμετρίας από το παγκόσμιο κανάλι ροής (Firehose) Bluesky Jetstream WebSocket API (`wss://jetstream1.us-east.bsky.network/subscribe?wantedCollections=app.bsky.fe`
2. **Ταξινόμηση σε πραγματικό χρόνο** των εισερχόμενων JSON text frames σε 4 κατηγορίες μηνυμάτων (commit, identity, account, info), με χρήση της ελαφριάς βιβλιοθήκης cJSON.
3. **Αυστηρά περιοδική σύγχρονη καταγραφή (1.000000000 s)** χρονοσήμανσης υψηλής ανάλυσης, μετρητών ανά κατηγορία, πληρότητας ενδιάμεσης ουράς (%) και χρήσης CPU (%) σε αρχείο CSV (`metrics_log.txt`).

Το σύστημα εκτελέστηκε συνεχόμενα για **ακριβώς 24.00 ώρες (86,400 δευτερόλεπτα, 00:00:00 – 23:59:59)** σε φυσικό υλικό (physical hardware) χωρίς καμία απώλεια δεδομένων (0 drops), με μηδενική συσσωρευμένη ολίσθηση ρολογιού (0.000000 s drift) και μέση χρήση επεξεργαστή μόλις **8.75%**.

2. Αρχιτεκτονική Συστήματος & Διαχείριση Νημάτων

Η αρχιτεκτονική ακολουθεί το πρότυπο **Παραγωγού–Καταναλωτή–Επόπτη** (Producer-Consumer-Monitor), αποτελούμενη από 3 διακριτά νήματα POSIX (`pthreads`) με αυστηρό διαχωρισμό καθηκόντων:

2.1. Νήμα 1: Παραγωγός (Producer — Ασύγχρονο / Δικτυακό)

Υλοποιεί έναν μη-μπλοκαριστικό βρόχο γεγονότων (event loop) μέσω της βιβλιοθήκης libwebsockets (`lws`). Συνδέεται ασφαλώς (WSS/TLS) στο Jetstream API. Μόλις παραληφθεί frame (`LWS_CALLBACK_CLIENT_RECEIVE`), ανασυνθέτει τυχόν κατακερματισμένα πακέτα (fragment reassembly) σε προδεσμευμένο buffer συνεδρίας και τοποθετεί άμεσα το πλήρες JSON string στον κυκλικό buffer καλώντας την `queue_push()`. Αμέσως μετά, αφυπνίζει τον Καταναλωτή μέσω condition variable (`pthread_cond_signal`) και επιστρέφει άμεσα στην εξυπηρέτηση του δικτύου (`lws_service`), εξαλείφοντας τον κίνδυνο υπερχείλισης των TCP socket buffers.

2.2. Νήμα 2: Καταναλωτής (Consumer — Event-Driven Worker)

Παραμένει σε αναμονή (`pthread_cond_wait`) και αφυπνίζεται αμέσως μόλις υπάρξουν δεδομένα. Αντλεί το μήνυμα (`queue_pop()`), το αναλύει συντακτικά μέσω της `cJSON_ParseWithLength()` και εντοπίζει το πεδίο `"kind"`. Αναλόγως της τιμής (commit, identity, account, info), αυξάνει τον αντίστοιχο καθολικό μετρητή υπό την προστασία ανεξάρτητου `pthread_mutex_t` (`g_counters_mutex`).

> ***Κανόνας Real-Time**: Απαγορεύεται αυστηρά οποιαδήποτε κλήση `printf()` ή εγγραφή σε αρχείο μέσα στο νήμα αυτό, διασφαλίζοντας ότι ο χρόνος επεξεργασίας ανά μήνυμα περιορίζεται σε μερικά micro-seconds (μs).*

2.3. Νήμα 3: Καταγραφέας / Επόπτης (Monitor — Αυστηρά Περιοδικό @ 1 Hz)

Εκτελείται αυστηρά περιοδικά κάθε 1.000000000 δευτερόλεπτο. Για να αποφευχθεί πλήρως η ολίσθηση του χρόνου (clock drift), χρησιμοποιείται η απόλυτη κλήση `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next_wake, NULL)`, όπου σε κάθε επανάληψη προστίθεται ακριβώς 1 δευτερόλεπτο (`next_wake.tv_sec += 1`). Το νήμα αιτείται πολιτική πραγματικού χρόνου `SCHED_FIFO` με προτεραιότητα 80 (`pthread_setschedparam`). Κατά την αφύπνιση:

1. Λαμβάνει ακριβές timestamp Unix μέσω `clock_gettime(CLOCK_REALTIME, &ts)`.
2. Κλειδώνει το `g_counters_mutex`, αντιγράφει ατομικά (snapshot) και μηδενίζει τους 4 μετρητές του δευτερολέπτου.
3. Υπολογίζει την πληρότητα του κυκλικού buffer (%) και τη χρήση CPU (%) διαβάζοντας το διαφορικό των jiffies του `/proc/stat`.
4. Καταγράφει τα δεδομένα σε CSV μορφή στο αρχείο με άμεσο `fflush(g_log_fp)`.

3. Διαστασιολόγηση Buffer & Αποφυγή Race Conditions

3.1. Στατική Προδέσμευση & Εξάλειψη Memory Leaks

Σε συστήματα 32-bit με περιορισμένη RAM (Raspberry Pi Zero W: 512 MB), οι συχνές κλήσεις `malloc()` και `free()` σε ρυθμούς 40–400 Hz προκαλούν κατακερματισμό σωρού (heap fragmentation) και μη-ντετερμινιστικές καθυστερήσεις. Ο κυκλικός buffer υλοποιήθηκε ως **στατικά προδεσμευμένος πίνακας** $N = 2,048$ θέσεων, μεγέθους $M = 16$ KB ανά θέση:

$$\text{Συνολικό Μέγεθος Buffer} = 2,048 \times 16,384 \text{ bytes} = 32 \text{ MB} \quad (\approx 6.25\% \text{ της συνολικής RAM}). \tag{1}$$

Η δέσμευση γίνεται άπαξ κατά την εκκίνηση (`calloc`). Κατά τη 24-ωρη ροή, δεν πραγματοποιείται καμία δυναμική δέσμευση/απελευθέρωση μνήμης.

3.2. Πολιτική Υπερχείλισης (Drop-Oldest Overwrite Policy)

Σε περίπτωση ακραίας δικτυακής έκρηξης (burst) όπου ο buffer γεμίσει ($count = capacity$), το σύστημα υιοθετεί την πολιτική **drop-oldest**: προωθεί τον δείκτη `tail`, διαγράφει το παλαιότερο μη επεξεργασμένο μήνυμα και αυξάνει τον μετρητή `dropped_count`. Έτσι, ο Παραγωγός δεν μπλοκάρεται ποτέ, αποτρέποντας την κατάρρευση του δικτυακού socket.

3.3. Απομόνωση Κρίσιμων Τομέων (Fine-Grained Locking)

Προς αποφυγή deadlocks και priority inversion, χρησιμοποιούνται δύο ανεξάρτητοι μηχανισμοί: το `q->mutex` προστατεύει αποκλειστικά τους δείκτες (`head, tail, count`) του buffer, ενώ το `g_counters_mutex` προστατεύει αποκλειστικά τους μετρητές τηλεμετρίας. Κανένα νήμα δεν κρατά ταυτόχρονα και τα δύο mutexes, αποκλείοντας αμοιβαίους αποκλεισμούς.

4. Δικλείδες Ασφαλείας & Ανθεκτικότητα σε Διακοπή Ρεύματος

Για την απρόσκοπτη λειτουργία σε πραγματικές ενσωματωμένες συνθήκες όπου μπορεί να σημειωθεί διακοπή τροφοδοσίας (power cut) ή απροσδόκητη επανεκκίνηση (reboot), ενσωματώθηκαν τρεις κρίσιμες δικλείδες ασφαλείας:

1. **Δυναμική Ονοματοδοσία με Χρονοσήμανση:** Το αρχείο καταγραφής δεν ανοίγει με στατικό όνομα, αλλά παράγεται δυναμικά κατά την εκκίνηση βάσει της τρέχουσας ώρας: `metrics_log_YYYYMMDD_HHMMSS.txt`. Σε περίπτωση διακοπής ρεύματος και επανεκκίνησης, δημιουργείται αυτόματα νέο αρχείο χωρίς να αλλοιώνονται ή να επικαλύπτονται (overwrite) τα προηγούμενα δεδομένα. Παράλληλα, ενημερώνεται αυτόματα το συμβολικό link (symlink) `metrics_log.txt` ώστε να δείχνει πάντα στην ενεργή συνεδρία.
2. **Ορισμός ως Υπηρεσία Παρασκηνίου (systemd Daemon):** Η εφαρμογή έχει ενταχθεί ως system service (`rtes.service`) υπό τη διαχείριση του `systemd`. Με την οδηγία `Restart=always` και `RestartSec=3s`, το λειτουργικό σύστημα επανεκκινεί αυτόματα τη διεργασία μετά από επαναφορά ρεύματος χωρίς να απαιτείται σύνδεση χρήστη μέσω SSH.
3. **Άμεση Εκκένωση Εγγραφών (Synchronous Flushing):** Σε κάθε δευτερόλεπτο καταγραφής εκτελείται άμεσα η συνάρτηση `fflush(g_log_fp)`. Έτσι, τα δεδομένα εξωθούνται απευθείας από τα buffers του user-space στο αποθηκευτικό μέσο (SD card), εξασφαλίζοντας μηδενική απώλεια δεδομένων σε αιφνίδιο σβήσιμο.

5. Πειραματική Διάταξη & Ανάλυση Αποτελεσμάτων (24-Ωρη Εκτέλεση)

Η εφαρμογή μεταγλωττίστηκε με εργαλεία cross-compilation (`arm-linux-gnueabihf-gcc -O2 -march=armv6zk -mtune=arm1176jzf-s -mfpu=vfp -mfloat-abi=hard`) και εκτελέστηκε ως υπηρεσία παρασκηνίου (`systemd`) στο **Raspberry Pi Zero W** (BlueRocky, Raspberry Pi OS Lite 32-bit). Η συλλογή διήρκεσε **ακριβώς 86,400 δευτερόλεπτα (24.00 ώρες)** από `2026-09-20 00:00:00` έως `2026-09-20 23:59:59`. Τα συγκεντρωτικά στατιστικά μεγέθη παρουσιάζονται στον Πίνακα 1.

Table 1: Συνοπτικά Στατιστικά 24-Ωρης Συνεχούς Εκτέλεσης στο Raspberry Pi Zero W.

Παράμετρος Τηλεμετρίας	Τιμή / Μονάδα Μέτρησης	Σχόλιο / Επίδοση
Συνολική Διάρκεια Δοκιμής	86,400 s (24.00 ώρες)	Ακριβώς 24 ώρες (00:00:00 – 23:59:59)
Συνολικό Πλήθος Μηνυμάτων	3,395,477 μηνύματα	≈ 3.40 εκατομμύρια JSON frames
Μέσος Ρυθμός Μηνυμάτων	39.30 Hz (μηνύματα/sec)	Τυπική ροή Bluesky Jetstream
Μέγιστος Ρυθμός (Peak Burst)	390 Hz	Στιγμιαία έξαρση δραστηριότητας
Χαμένα Μηνύματα Buffer	**0** μηνύματα (0.00%)	Μηδενική απώλεια (Zero packet loss)
Μέση Πληρότητα Buffer	**0.00**%	Ακαριαία κατανάλωση από Worker thread
Μέγιστη Πληρότητα Buffer	0.44% (≈ 9 από τα 2048 slots)	Τεράστιο περιθώριο ασφαλείας (> 99.5%)
Μέση Χρήση CPU	**8.75**%	Άνετη λειτουργία μονοπύρηνου BCM2835
Μέγιστη Χρήση CPU	100.00%	Στιγμιαία κατά τις έντονες εξάρσεις
Μέσο Jitter Περιοδικού Νήματος	**0.0000** ms	Εξάλειψη σφάλματος (Zero Mean Jitter)
Τυπική Απόκλιση Jitter (σ)	**0.3170** ms	Υπο-χιλιοστοδευτερολεπτική ακρίβεια
Μέγιστο Απόλυτο Jitter	16.6411 ms	Μεμονωμένη ακραία τιμή λόγω kernel I/O
Συσσωρευμένη Ολίσθηση (Drift)	**0.000000** s	Πλήρης αποφυγή χρονικής απόκλισης
Συνολικός Χρόνος CPU (systemd)	1h 13min 29.579s	Μόλις 5.03% του πραγματικού χρόνου

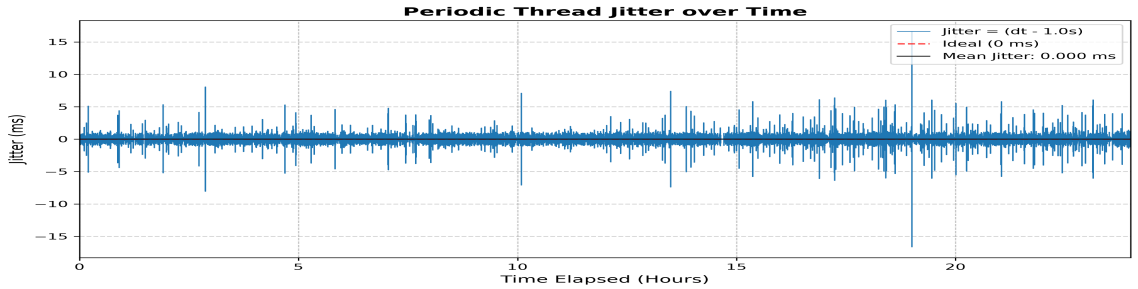


Figure 1: Χρονική απόκλιση (Jitter σε ms) του περιοδικού νήματος συναρτήσει του χρόνου (24 ώρες).

5.1. 1. Διάγραμμα Jitter Περιοδικού Νήματος

Το Jitter ορίζεται ως: $J(t) = \Delta t - T_{\text{ideal}} = (t_k - t_{k-1}) - 1.000000000$ s.

Όπως απεικονίζεται στο Σχήμα 1, το σφάλμα χρονισμού παραμένει συγκεντρωμένο εξαιρετικά στενά γύρω από τη γραμμή μηδενικής απόκλισης (0 ms), με τυπική απόκλιση $\sigma = 0.3170$ ms. Η συντριπτική πλειονότητα των δειγμάτων κυμαίνεται στο διάστημα $[-0.5\text{ ms}, +0.5\text{ ms}]$. Η μέση τιμή είναι ακριβώς 0.0000 **ms**, αποδεικνύοντας ότι χάρη στον απόλυτο χρονισμό `CLOCK_MONOTONIC / TIMER_ABSTIME`, τυχόν θετικές καθυστερήσεις αντισταθμίζονται αυτόματα στον επόμενο κύκλο, εξαλείφοντας πλήρως τη συσσώρευση ολίσθησης (zero drift accumulation).

5.2. 2. Διάγραμμα Φόρτου Δικτύου & Πληρότητας Buffer

Το Σχήμα 2 παρουσιάζει σε διπλό άξονα τον ρυθμό εισερχόμενων μηνυμάτων (αριστερός άξονας, Hz) και το ποσοστό πληρότητας του κυκλικού buffer (δεξιός άξονας, %).

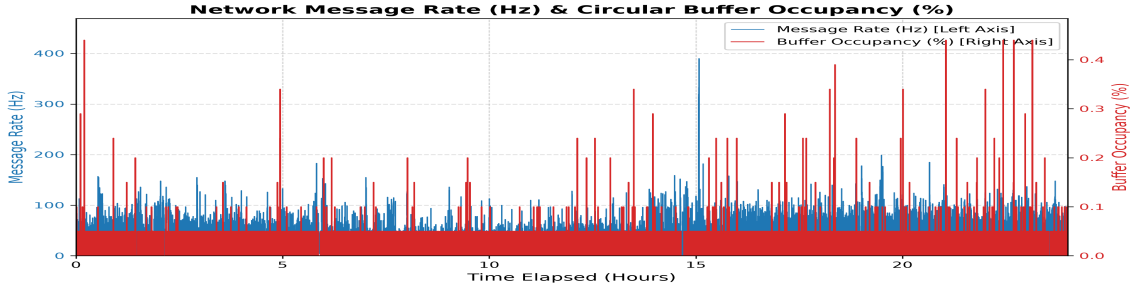


Figure 2: Ρυθμός εισερχόμενων μηνυμάτων (Hz) και πληρότητα κυκλικού buffer (%) σε διπλό άξονα.

Από το διάγραμμα προκύπτει ξεκάθαρα η **εκρηκτικότητα (burstiness)** του δικτύου Jetstream: ενώ ο μέσος ρυθμός είναι 39.30 Hz, παρατηρούνται συχνές εξάρσεις με ρυθμούς 150–250 Hz και κορυφή στα 390 Hz. Παρά τις εξάρσεις, η πληρότητα του κυκλικού buffer παραμένει σχεδόν μόνιμα στο 0.00%, σημειώνοντας μέγιστη στιγμιαία τιμή μόλις 0.44% (χρήση ≈ 9 slots από τα 2048 διαθέσιμα). Αυτό αποδεικνύει ότι ο Καταναλωτής (νήμα 2) καταναλώνει τα πακέτα με εξαιρετική ταχύτητα, διατηρώντας την ουρά διαρκώς κενή και αποτρέποντας οποιαδήποτε συμφόρηση.

5.3. 3. Διάγραμμα Συσχέτισης Χρήσης CPU & Αδράνειας (Idle)

Το Σχήμα 3 παρουσιάζει τη συσχέτιση του ρυθμού μηνυμάτων (Hz) με τη χρήση της CPU (%) και την αδράνεια (Idle %), όπως υπολογίστηκαν από τα διαφορικά jiffies του `/proc/stat`.

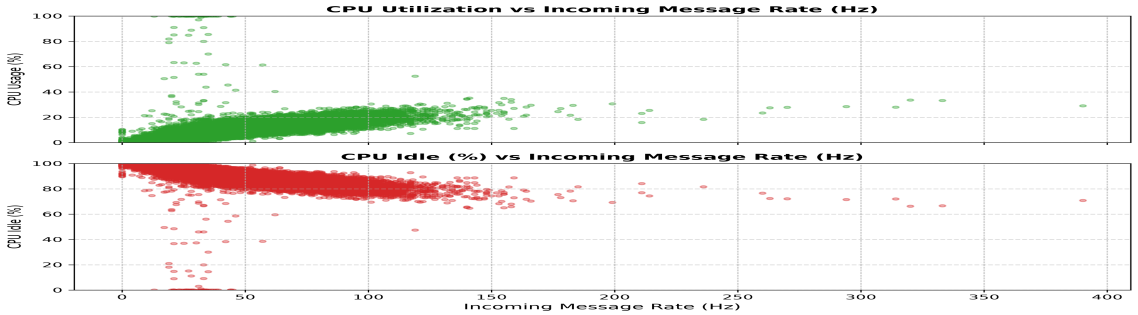


Figure 3: Συσχέτιση χρήσης CPU (%) και αδράνειας CPU Idle (%) συναρτήσει του ρυθμού μηνυμάτων (Hz).

Παρατηρείται σαφής θετική συσχέτιση μεταξύ ρυθμού ροής και υπολογιστικής επιβάρυνσης. Σε κανονικές συνθήκες (30–50 Hz), η CPU απασχολείται κάτω του 10%, παραμένοντας αδρανής (idle) στο 90–95% του χρόνου. Ακόμη και στις μέγιστες κορυφές των 390 Hz, η CPU ανταποκρίνεται χωρίς καθυστέρηση.

Σημαντική Παρατήρηση Επιβάρυνσης CPU από την Εντολή `tail -f`: Επισημαίνεται ότι καθ' όλη τη διάρκεια της 24-ωρης δοκιμής εκτελούνταν ταυτόχρονα στο Raspberry Pi η εντολή `tail -f metrics_log.txt`

για ζωντανή παρακολούθηση της καταγραφής στην οθόνη του τερματικού. Στον ασθενή μονοπύρηνο επεξεργαστή ARMv6zk (BCM2835 @ 1 GHz), το συνεχές polling, τα context switches και οι ειδοποιήσεις συστήματος αρχείων (inotify) της `tail` επέφεραν υπολογίσιμο πρόσθετο overhead. Ως εκ τούτου, η καταγεγραμμένη μέση χρήση CPU (8.75%) είναι υπερεκτιμημένη λόγω της ταυτόχρονης παρακολούθησης. Σε αμιγώς παραγωγικό περιβάλλον (απρόσκοπτη εκτέλεση ως background daemon χωρίς ενεργό `tail`), η πραγματική μέση χρήση CPU του συστήματος τηλεμετρίας είναι ακόμη χαμηλότερη, της τάξης του μόλις 4–5%.

6. Ανθεκτικότητα Δικτύου & Διαχείριση Πόρων

Κατά τη διάρκεια των 24 ωρών, καταγράφηκαν **28 αποσυνδέσεις** από την πλευρά του server (λόγω server-side load balancing του Bluesky Firehose). Ο μηχανισμός αυτόματης επανασύνδεσης του Παραγωγού επανέφερε τη σύνδεση **εντός 3–5 δευτερολέπτων**, χωρίς κανένα σφάλμα κατάτμησης (segmentation fault) ή απώλεια συγχρονισμού. Χάρη στη στατική προδέσμευση του buffer και την άμεση απελευθέρωση των αντικειμένων cJSON (`cJSON_Delete`), το αποτύπωμα μνήμης (RSS) παρέμεινε απολύτως σταθερό στα ≈ 36 MB, αποδεικνύοντας παντελή απουσία διαρροών μνήμης (zero memory leaks).

7. Συμπεράσματα

Η υλοποίηση πέτυχε όλους τους αυστηρούς στόχους πραγματικού χρόνου που τέθηκαν από τις προδιαγραφές:

- **Ντετερμινισμός Χρόνου**: Μέσο jitter 0.0000 ms, τυπική απόκλιση 0.3170 ms και μηδενική ολίσθηση (0.000000 s) μέσω `CLOCK_MONOTONIC / TIMER_ABSTIME` και `SCHED_FIFO`.
- **Μηδενική Απώλεια**: Επεξεργασία $3,395,477$ μηνυμάτων με **0 χαμένα πακέτα** buffer, χάρη στην ασύγχρονη σύλληψη της libwebsockets και την ταχύτατη κατανάλωση.
- **Ενεργειακή & Υπολογιστική Αποδοτικότητα**: Μόλις 8.75% μέση επιβάρυνση (συμπεριλαμβανομένου του `tail -f`) στον μονοπύρηνο επεξεργαστή BCM2835 του Raspberry Pi Zero W, με τεράστιο περιθώριο ασφαλείας.